

# Peer2Peer Systems and Blockchains

## Final Project A.A. 2025/26

### Decentralised Lending Service

Version 1.1 - 11th of May 2026

#### 1. Service description

The goal of this project is to implement a decentralised service running on Ethereum that allows a decentralised pooling of funds that can be loaned by other users. The service operates with a common **pool** where **contributors** may deposit or withdraw value, and loan **applicants** may borrow value from. The decision over which loans to fund is taken democratically by all contributors proportionally to their current available contribution to the pool. Moreover, the service relies on a trusted centralised oracle to read Bitcoin address balances to be used as proof of liquidity when requesting a loan.

Funds from the common pool (deposited or withdrawn by their contributor owners) may be locked by approved loans. Applicants may submit requests for new loans but the contributors are the ones in charge of voting to accept or reject a new loan proposal. This voting process is proportional, for each contributor, to the amount of funds not yet locked by a previous loan in the common pool they own. Applicants are required to show the sustainability of their loan by showing that there exists an address in the Bitcoin blockchain holding enough funds as to cover the loan<sup>1</sup>. Once a loan has been granted, the corresponding amount is locked in the common pool and the loan can be later repaid with interests in separate installments. To protect contributors from loans that do not get repaid, part of all loan interests are collected in a compensation pool that can be employed by contributors of failed loans to recoup their losses.

The system is implemented through a bundle of smart contracts, accompanied by an off-chain oracle to implement a very basic cross-chain protocol to write aggregated information about the Bitcoin blockchain (up to date address available cumulative balances) in Ethereum.

#### 1.1 Actors

Users may interact with the system either as contributors or applicants (the same user can cover both roles at the same time). Any user that has a non zero amount deposited in the funding pool (including value currently locked in loans) is considered a contributor. Any user with an active loan proposal, an active loan, or a granted/denied loan is considered an applicant.

---

<sup>1</sup> This is a simplification for didactic purposes, it is NOT at all secure in any way. First because the applicant is not required to prove ownership of the Bitcoin funds shown as collateral, so they may indicate other users' funds. Moreover, the funds are not required to be locked somehow, so they could be later spent once a loan is granted but before it is repaid.

A contributor deposits and withdraws value to/from the funding pool, accrues interests over successful loans, may be compensated for failed loans from the compensation pool, and may vote to accept or reject new loan proposals.

An applicant may propose a new loan (specifying amount, interest rate, and duration), execute an approved loan, and repay a successful loan.

## 1.2 Pools management

The service manages at least two pools: the **funding pool** and the **compensation pool**.

The funding pool contains all value deposited by contributors. Part of this value may be currently locked inside loans. Any contributor may deposit new value to the funding pool, or withdraw any quantity of their deposited value not currently locked in a loan.

The compensation pool contains part of the value accrued as interest by past loans. New value is added to this pool after each repaid loan, and value is withdrawn to cover for failed loans.

## 1.3 Operations

Contributors may do the following operations.

- Deposit value to the funding pool. Any user is allowed to deposit any amount greater than the minimum deposit amount (constant value fixed).
- Withdraw value from the funding pool. The withdrawn amount can not be greater than the current value deposited by that user in the funding pool that is NOT currently locked into a loan. This value (different from user to user and over time) is henceforth named **disposable value**.
- Vote for an active loan proposal. The vote is a boolean approve/reject value and can only be cast for an active loan proposal. Not voting equates to an implicit reject vote.
- Ask for compensation for a failed loan. A loan is considered **failed** if the loan amount was not fully repaid and the current time is greater than the expiration date. Partially repaid loans that are expired are considered failed. Compensation can only be requested for a loan that failed. With the request, the contributor receives from the compensation pool enough value as to recoup the value locked by them in the loan not yet repaid by the failed loan applicant (interests paid by partially repaid loans are not considered in this computation). The contributor may receive less than owed if not enough value is left in the compensation pool. This operation can be performed multiple times by the same **contributor** for the same failed loan to try to recoup value as the compensation pool gets refilled by other loans. Asking for a compensation proportionally forfeits the applicant repayments in case the applicant later refunds part of the failed loan. A compensation also marks that loan as failed, influencing the global collateral percentage accordingly. A loan can be marked as failed only

once. A failed loan can not later become successful even if the amount were to be later repaid in full.

Applicants may do the following operations.

- Request the recording of a Bitcoin address liquidity by the **Bitcoin liquidity oracle** (see below).
- Submit a new loan proposal. The applicant specifies amount, interest rate (i.e., percentage of the specified amount that will be credited to contributors that backed the loan, this value is expressed as a natural number between 1 and 100 included), duration, and a Bitcoin address to be used as proof of liquidity.
- Request the resolution of a loan proposal. A resolution can be requested only for loan proposals that have been proposed for longer than the proposal voting period. A resolution can only be requested by the original applicant of that loan proposal. If the cumulative disposable value currently in the funding pool is lower than the loan requested amount, the proposal is rejected (and closed as if a majority reject vote won). If the **Bitcoin liquidity check** fails (see below), the proposal is rejected (and closed as if a majority reject vote won). The resolution process evaluates if the proposal passed by counting the weighted approval votes received by the proposal from contributors. The votes are weighted over the current disposable value for each contributor compared to the cumulative disposable value currently in the pool. Contributors that have not yet cast a vote are considered as reject. If the weighted majority vote is reject or it is an exact tie between accept and reject, the proposal is closed (no more votes can be received, no further resolution requests may be performed). If the result is approve, the request is closed and a new **loan management contract** is created, managing the loan locked value as a new **active loan**. Value for a loan is locked from all contributors' value, proportionally to their current disposable value. E.g., if the current disposable value is 10 for contributor 1, 20 for contributor 2 and 30 for contributor 3, and the loan proposal amount is 12, 2 value will be locked from contributor 1, 4 value from contributor 2, and 6 value from contributor 3. Any leftover value discrepancy due to finite precision arithmetic will be deducted from the loan amount and not credited to the applicant (the loaned amount MUST reflect this discrepancy rather than considering the proposal amount value). The new active loan is set with the current collateral percentage.
- Partially repay a loan. An applicant may partially repay one of their active or failed loans. Only the original applicant may repay one of their loans. An applicant may repay a loan even more than the original loan amount (even though it may make little economical sense). The value credited through a loan repayment is split in its interest and base amount components. The base amount is split among the participants by refunding the contributor funding pool positions in order of locked value, from initial highest to lowest (when the loan was created, not accounted for partially repaid loan amounts so far) breaking any tie by contributor addresses ordering. Any amount exceeding the original loan amount (i.e., after all contributors have been repaid) is credited to the compensation pool instead. The interest is split between **gain** and **collateral** according to the collateral percentage of this loan. The gain is equally credited to the contributors directly (not to the funding pool) proportionally

with the same rule (any leftover value discrepancy due to finite precision arithmetic is credited to the compensation pool instead). All collateral is credited to the compensation pool. Note that even if the operation is called a partial repayment, the applicant may repay the entire loan in a single operation by using this rule. If all value (or more) is repaid by this operation, the loan is marked as successful, influencing the global collateral percentage accordingly, and is closed.

Notes:

- all time periods must be measured in block height difference. E.g., a time of 10 means a block height difference between the current block and original block of 10.

Constants:

- proposal voting period: minimum amount of time that a loan proposal can be voted upon by contributors (as a loan proposal can not be resolved before this period). Fixed at 12;
- minimum deposit amount: minimum value of a deposit operation. Fixed at 100 000 wei;
- initial collateral percentage: initial collateral percentage. Fixed at 50. The current collateral percentage value is between 1 and 100. The value is increased by 5 for every failed loan and decreased by 5 for every successful loan;
- Bitcoin liquidity oracle address: the address of the smart contract acting as on-chain endpoint for the Bitcoin liquidity oracle.
- Bitcoin liquidity oracle minimum fee: minimum fee for a Bitcoin address balance update request. Fixed as the gas cost of the oracle contract update operation (to be computed depending on the provided implementation) times 0.1 gwei.
- BTC/ETH exchange rate: fixed at 1 BTC = 30 ETH.

## 1.4 The Bitcoin liquidity oracle

It is required to implement a centralised trusted oracle service that may read the bitcoin blockchain, compute the current available balance of a Bitcoin address, and save this value into a smart contract on a target Ethereum chain.

- The on-chain component (on the Ethereum chain) of the oracle is a smart contract that remembers the available balance (in BTC) of a set of Bitcoin addresses.
- The oracle smart contract advertises a function to receive a request to update the balance of a given Bitcoin address (or add a new association if the address is not already present in the contract). This function requires the requester to pay a fee (greater or equal to the constant **minimum oracle fee** value) for the request to be valid.
- The oracle serves all new requests (we assume it to be trusted also in the sense of not dropping intentionally any request) by querying the up to date Bitcoin blockchain, computing the available balance for the target address (i.e., the sum of all its unspent transaction outputs), and updating the value in the oracle contract.
- The lending service interacts with the oracle in two ways:

- applicants should (but are not required to) request the oracle to update the balance of the Bitcoin address they plan to use as liquidity proof in a new loan proposal request;
- during a loan proposal resolution request the Bitcoin liquidity check is performed. This check consists in reading from the oracle contract the balance of the Bitcoin address associated with the current loan proposal, convert this amount in ether by using the constant BTC/ETH exchange rate, check if the resulting balance is greater than the requested loan amount. If the balance is equal or higher than the requested amount, the check is passed, otherwise it is failed. Note that there is no signature or other form of ownership proof required with this check. So the applicant may pick any Bitcoin address, no matter whether they are the owner or not. This has clearly no sense in practice, it is a solution adopted in this project only for didactic purposes.
- IT IS REQUIRED to implement the off-chain components of the oracle as if new Bitcoin blocks were generated as normal, so the code should be designed to elaborate one new block at a time. HOWEVER, do run this code on the first 131 000 blocks only of the Bitcoin main net blockchain (as seen during the lecture examples). This is to avoid long sync times and large file sizes.

## 1.5 Implementation requirements

- Smart contracts
  - Events must be emitted as needed (E.g., when a new proposal is submitted).
  - A new active loan must be managed by a newly deployed dedicated smart contract.
  - Properly manage contracts “termination” and main contract upgradability.
- Testing and deployment
  - Use hardhat for testing. Provide a set of source files to test and showcase the proper execution of the contracts developed under different circumstances and user behaviours.
  - Use a local private chain based on the genesis file provided. The prefunded accounts provided can ONLY be used to transfer value to other accounts (e.g., new one created), they can not be used to deploy contracts or execute other types of transactions.
  - Provide a python script for executing the required initial setup of the service (including the creation of any new account required).
  - Provide a python script for executing an example set of operations showcasing the correct execution of the service (including the creation of any new account required). This script must include printing the balance of the involved accounts after any meaningful change, as well as any other relevant state variable.
  - Provide a python script that implements an automated strategy for a contributor that always votes to approve any new loan proposal (even if the vote may be

irrelevant if the entirety of the contributor funds are locked in active loans). This requires for the contributor to “notice” when new loan proposals are made.

- Provide a set of tests or scripts to measure each operation gas cost (as seen during the lectures).
- Show how to modify the smart contracts to introduce a reentrancy vulnerability and show a test performing a reentrancy attack on the modified code (and any ad hoc additional malicious contract).
- Discuss if there is a possible malicious strategy for a contributor to gain an unfair remuneration at the expense of other honest contributors (without relying on a reentrancy attack), e.g., by leveraging the loan repayment majority rule.

## 2. Submission rules

The required material to be submitted for evaluation is:

- all smart contracts Solidity source code;
- all scripts used for testing and deployment (as specified in section 1.5);
- a zip of the hardhat folder used during testing;
- a pdf report of no more than 5 pages containing:
  - a brief description of the main choices made during the implementation of the smart contracts;
  - the gas evaluation results, reentrancy modification, malicious strategies discussion as detailed in section 1.5;
  - a brief user manual with the instructions to use the provided code.

All material (properly organised in an intelligible folder structure) should be submitted in a single zip file electronically on Moodle. The project will be discussed during the oral exam (that will include both a project discussion and topics presented during the lectures) of the first available session the student enrolled in. The student may use their personal laptop during the project discussion at the oral exam. The project must be developed by a group of at most 2 students, but each student will still be required to be able to discuss their project alone, whether it was made in a group or not.